Puntatori in C

Classe 3TRED — Liceo Scientifico Scienze Applicate

Indice degli argomenti

2. Il concetto di puntatore

3. Puntatori e array

2. Il concetto di puntatore

Cos'è la memoria (in breve)

Ogni variabile che dichiariamo viene salvata da qualche parte nella **RAM**.

Ogni "casella" di memoria ha:

- un **indirizzo** (un numero che la identifica, come il numero civico di una casa)
- un **contenuto** (il valore salvato in quella casella)

```
Indirizzo:   0x7ffee3a1c05c
Contenuto:   42
```

Finora, per usare una variabile, abbiamo sempre pensato al suo **valore** (42).
Un puntatore ci permette invece di lavorare con il suo **indirizzo**.

Definizione di puntatore

> Un **puntatore** è una variabile che **non contiene un valore "normale"**, ma **contiene l'indirizzo di memoria** di un'altra variabile.

Se ho una variabile:

```
int x = 42;
```

`x` è come una scatola etichettata che contiene il valore 42.

Un puntatore a `x` è un'altra variabile che, invece del valore 42, contiene **l'indirizzo della scatola x**.

📌 Pensala così: il puntatore non è "il numero 42", è **"dove si trova"** il numero 42 in memoria.

I due operatori fondamentali

Operatore	Nome	Significato
`&`	"indirizzo di"	Restituisce l'indirizzo di una variabile
`*`	"valore puntato da" (dereferenziazione)	Restituisce il contenuto presente all'indirizzo memorizzato nel puntatore

```
int x = 42;
int *p;      // p è un puntatore a intero (per ora non punta a niente di valido)

p = &x;      // ora p contiene l'indirizzo di x
```

⚠️ Attenzione: `*` ha **due significati diversi** in C:

- nella **dichiarazione** (`int *p;`) indica "p è un puntatore"
- nell'**uso** (`*p`) indica "vai a leggere/scrivere il valore all'indirizzo contenuto in p"

Esempio: dichiarare e usare un puntatore

```c
#include <stdio.h>

int main(void) {
    int x = 42;
    int *p;       // dichiaro un puntatore a intero

    p = &x;       // p ora "punta a" x, cioè contiene l'indirizzo di x

    printf("Valore di x:            %d\n", x);
    printf("Indirizzo di x (&x):    %p\n", (void*)&x);
    printf("Valore di p (indirizzo):%p\n", (void*)p);
    printf("Valore puntato da p:    %d\n", *p);  // dereferenziazione -> 42

    return 0;
}
```

`%p` è il format specifier per stampare un **indirizzo di memoria** (va sempre convertito a `void*`).

Output tipico (gli indirizzi variano ad ogni esecuzione!)

```
Valore di x:              42
Indirizzo di x (&x):      0x7ffee3a1c05c
Valore di p (indirizzo): 0x7ffee3a1c05c
Valore puntato da p:      42
```

✅ Nota che **l'indirizzo di x** e **il valore contenuto in p** sono **identici**: è proprio questo il senso di "p punta a x".

Modificare una variabile tramite puntatore

```c
#include <stdio.h>

int main(void) {
    int x = 42;
    int *p = &x;

    printf("Prima:  x = %d\n", x);

    *p = 100;    // modifico il valore puntato da p -> modifico direttamente x!

    printf("Dopo:   x = %d\n", x);

    return 0;
}
```

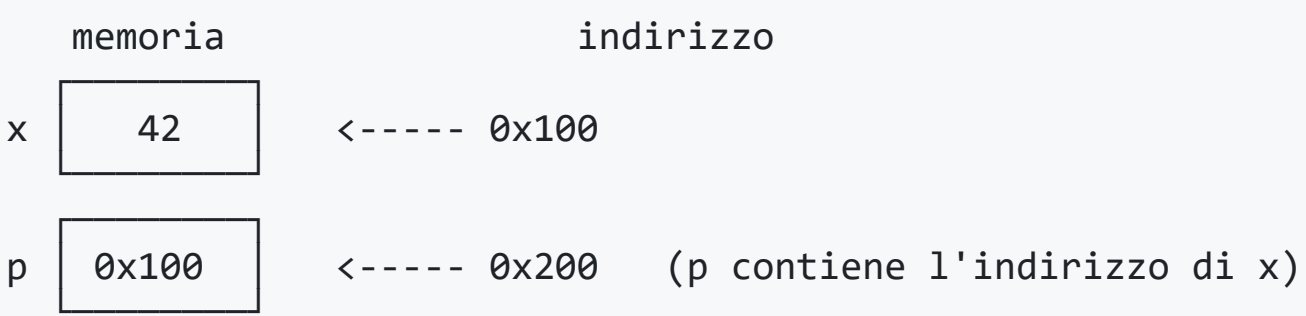
Output:

```
Prima:  x = 42
Dopo:   x = 100
```

💡 `*p = 100` **non cambia dove punta p**, cambia il **contenuto della cella di memoria** puntata da

Riassunto visivo

```
int x = 42;
int *p = &x;
```



- `x` → 42
- `&x` → 0x100 (indirizzo di x)
- `p` → 0x100 (perché p = &x)
- `*p` → 42 (vado all'indirizzo 0x100 e leggo il contenuto)

3. Puntatori e Array

Il nome di un array è (quasi) un puntatore

In C, quando usi il nome di un array senza indice, questo viene automaticamente convertito nell'**indirizzo del suo primo elemento**.

```c
int numeri[5] = {10, 20, 30, 40, 50};
int *p = numeri;      // equivalente a: int *p = &numeri[0];

printf("%p\n", (void*)numeri);     // indirizzo del primo elemento
printf("%p\n", (void*)&numeri[0]); // stesso indirizzo!
printf("%d\n", *p);                // 10 (primo elemento)
```

⚠️ Attenzione: `numeri` **non è una variabile puntatore** modificabile come `p` (non puoi scrivere `numeri = ...`), ma **si comporta come un puntatore costante** al primo elemento.

Aritmetica dei puntatori

Sommare 1 a un puntatore **non aggiunge 1 byte**, ma fa avanzare il puntatore di `sizeof(tipo)` byte, cioè lo sposta **all'elemento successivo**.

```c
int numeri[5] = {10, 20, 30, 40, 50};
int *p = numeri;

printf("%d\n", *p);       // 10  (numeri[0])
printf("%d\n", *(p + 1)); // 20  (numeri[1])
printf("%d\n", *(p + 2)); // 30  (numeri[2])

p++;                       // ora p punta a numeri[1]
printf("%d\n", *p);       // 20
```

📌 In generale: `numeri[i]` è **equivalente** a `*(numeri + i)`.

Scorrere un array con puntatori vs indici

```c
#include <stdio.h>

int main(void) {
    int numeri[5] = {10, 20, 30, 40, 50};

    // Metodo classico (con indice)
    for (int i = 0; i < 5; i++) {
        printf("%d ", numeri[i]);
    }
    printf("\n");

    // Metodo con puntatori
    int *p = numeri;
    for (int i = 0; i < 5; i++) {
        printf("%d ", *(p + i));
    }
    printf("\n");

    return 0;
}
```

Entrambi i cicli stampano lo stesso risultato: `10 20 30 40 50`.

6. Errori comuni e cose da NON fare

(con puntatori e con array)

Errore 1: puntatore non inizializzato ("wild pointer")

```
int *p;         // p contiene un indirizzo A CASO (memoria non inizializzata)
*p = 10;        // ERRORE GRAVE: sto scrivendo in una zona di memoria
                // che non conosco e che non mi appartiene!
```

- Il programma **potrebbe** funzionare per caso, **potrebbe** andare in crash (*segmentation fault*), o **potrebbe** corrompere altri dati: è **comportamento indefinito**
- ✅ Corretto: inizializza sempre il puntatore, o fallo puntare a `NULL` finché non è pronto

```
int *p = NULL;      // esplicitamente "non punta a niente"
if (p != NULL) {    // controllo prima di usarlo
    *p = 10;
}
```

Errore 2: assegnare un numero a caso come indirizzo

```
int *p;
p = 1000;       // SBAGLIATO: 1000 non è un indirizzo valido gestito dal tuo programma
*p = 5;         // quasi certamente CRASH (segmentation fault)
```

- Un indirizzo valido si ottiene **solo** con `&variabile`, con `malloc`, o dal nome di un array
- Il compilatore in genere segnala un **warning/errore di tipi incompatibili** se provi a scrivere codice simile

Errore 3: confondere il puntatore con il valore puntato

```
int x = 10;
int *p = &x;

p = 20;       // SBAGLIATO: sto dicendo "fai puntare p all'indirizzo 20"
              // (quasi certamente non valido!) -> errore di compilazione/warning

*p = 20;      // CORRETTO: modifico il VALORE puntato da p, cioè modifico x
```

📌 Dimenticare la `*` è **l'errore più frequente** quando si iniziano a usare i puntatori: `p` è l'indirizzo, `*p` è il contenuto.

Errore 5: `&x` non è un contenitore in cui scrivere

```
int x = 10;

&x = 20;   // ERRORE DI COMPILAZIONE
           // &x è solo un valore calcolato (l'indirizzo di x),
           // non una "scatola" in cui posso mettere qualcosa
```

`&x` si può **leggere** (usare come valore), ma non è una variabile: non puoi assegnargli nulla, così come non puoi scrivere `(3 + 2) = 10;`.